

Applied Reinforcement Learning (A2C) on Transportation Stocks

handed in by
Christopher Voizard

October 10, 2026

Contents

1	Introduction and Companies Selected	3
1.1	Share price over the last 10 years	4
1.2	Price-earnings ratio (P/E ratio)	5
1.3	Price-to-sales ratio (P/S ratio), last 12 months	5
1.4	Log-returns	6
2	A2C Algorithm Applied and Analysis	6
2.1	Boeing	6
2.2	Airbus	7
2.3	Toyota	8
3	Technical Indicators	9
3.1	Relative Strength Index (RSI)	9
3.2	Moving Average Convergence Divergence (MACD)	9
3.3	On-Balance Volume (OBV)	9
3.4	Aroon Indicator	10
3.5	Stochastic Oscillator	10
3.6	Comments	11
4	Correlated State Variables	11
5	Test Trained Agent on Different Time Periods	12
6	Overall Findings and Limitations	13
7	Summary on One Selected Paper	14
7.1	Deep Robust Reinforcement Learning for Practical Algorithmic Trading	14
7.2	Related Work: Three Answers to the Generalization Problem	14
8	2026 Update: Fundamentals and a Re-run of the A2C Part	15
8.1	Updated Fundamental Figures	15
8.2	Re-run of the A2C Part on New Data	16
8.3	Deviations from the Original Notebook	17
8.4	Conclusion of the Update	18
9	A3C: A Second Algorithm, a Decomposition and a Generative Market Model	18
9.1	The Baseline Has No Edge	19
9.2	Decomposition: Which Factor Carries the Effect?	19
9.3	A Generative Market Model Instead of Rolling Windows	19
9.4	Isolation: the Bootstrap Carries, the Two Passengers Do Not	20
9.5	What the Overfitting Picture Shows	21
9.6	Robustness Check: the Other Two Stocks	21
9.7	Generalization Check: Six Further Stocks	22
9.8	Limitations	24
9.9	Conclusion of the A3C Part	24
A	Notebook Code	25

1 Introduction and Companies Selected

As the subject of this study, I chose three companies more or less related to the transportation industry, which for various reasons have had difficulties in recent years, especially environment-related, to publish good business figures. The two aircraft manufacturers **Boeing** and **Airbus**, which share the market completely between themselves, were particularly affected by the lockdown during the last pandemic and the shutdown. However, vehicle manufacturers such as market leader **Toyota** are also facing changing market demand. In view of these challenges, it is interesting to conduct an analysis of future performance and to compare buy-and-hold strategies to our trained agent.

The following descriptive statistics are used to evaluate the performance of the shares:

- share price over the last 10 years,
- price-earnings ratio (P/E ratio),
- price-to-sales ratio (P/S ratio),
- log-returns.

Price-earnings ratio (P/E ratio). The price/earnings ratio is the number of years in which the company would have earned its stock market value if profits had remained constant. In the calculation, the current share or stock market price is divided by the company's earnings per share. The company's earnings per share are calculated by dividing the company's earnings by the total number of shares. The common and relevant sites give a guideline value between 12 and 15 for the interpretation of the P/E ratio. If the results are below this value, a share tends to be considered cheap; if the results are above 15, the share may tend to be considered more expensive. The formula is

$$P/E = \frac{\text{share price}}{\text{earnings per share}}. \quad (1)$$

Price-to-sales ratio (P/S ratio). The price-to-sales ratio (P/S ratio) is a share ratio that shows the market value of a share in relation to its annual sales. The lower the P/S compared to the values of other companies in the same industry, the more favorable the respective share is. The common and relevant sites give a guideline value above 1 and below 1 for the interpretation of the P/S ratio. If the results are below this value, a share tends to be considered cheap; if the results are above 1, the share may tend to be considered more expensive. It can be calculated as follows:

$$P/S_{total} = \frac{\text{sales}}{\text{market capitalisation}}. \quad (2)$$

Log-returns. The log return can be calculated by simply taking a natural logarithm of the current price (p_1) divided by the initial price (p_0):

$$r_1 = \ln\left(\frac{p_1}{p_0}\right). \quad (3)$$

1.1 Share price over the last 10 years

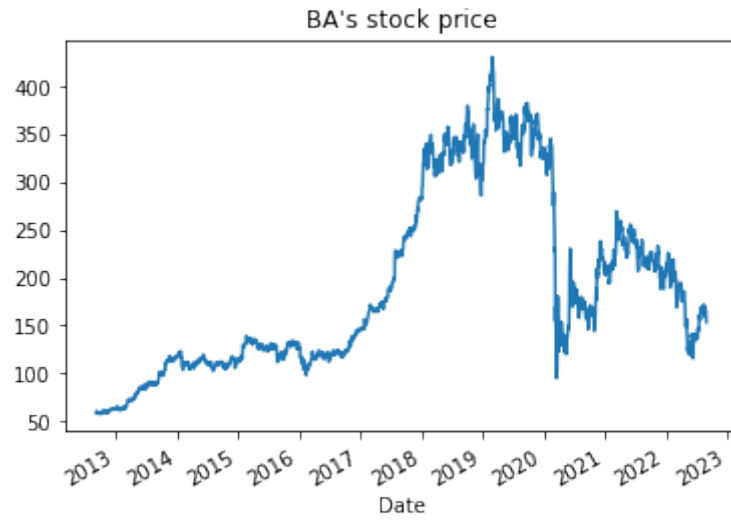


Figure 1: Boeing (BA) – closing price over the last 10 years.



Figure 2: Airbus (AIR.PA) – closing price over the last 10 years.

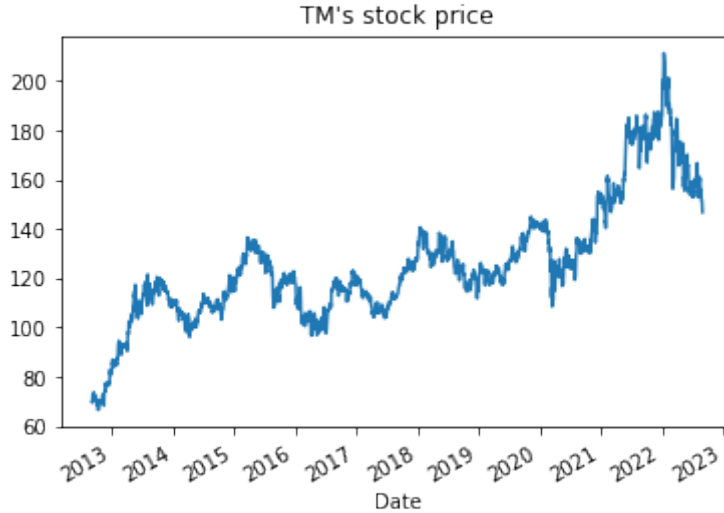


Figure 3: Toyota (TM) – closing price over the last 10 years.

Comments. The price chart shows the time sequence on the abscissa and the price of the share on the ordinate. Both Boeing and Airbus show a significant drop in price at the beginning of 2020, i.e. the first lockdown and the suspension of air traffic. While Airbus was almost back to its previous high of 2020 at the beginning of 2022, Boeing shows a much more negative trend. Toyota, on the other hand, reached its peak at the beginning of 2022, but has also shown a negative trend since then.

1.2 Price-earnings ratio (P/E ratio)

The P/E values obtained for the three companies are summarised in Table 1.

Table 1: Price-earnings ratio (P/E).

Company	P/E	Remark
Boeing	–	value not returned by the data source
Airbus	19.89	
Toyota	10.37	

Comments. Given the P/E values for Airbus and Toyota of 19.9 and 10.4, respectively, it can be seen that Airbus is above the guideline value of 15 and can thus be regarded as rather expensive, while Toyota can be regarded as rather inexpensive by industry standards.

1.3 Price-to-sales ratio (P/S ratio), last 12 months

Table 2: Price-to-sales ratio (P/S).

Company	P/S
Boeing	1.502
Airbus	1.458
Toyota	0.006

Comments. Similar to the P/S value above, it can be seen that both Boeing and Airbus are classified as more or less expensive with values of 1.5 and 1.4 respectively, while Toyota could

even be classified as favorable with a value close to zero.

1.4 Log-returns

Table 3: Cumulative log-returns over the observed period.

Company	Log-return	in %
Boeing	-0.3025	-30.2
Airbus	-0.1451	-14.5
Toyota	-0.1486	-14.9

Comments. The log return of the last year shows for all three stocks that a simple buy-and-hold strategy would have led to more or less larger losses during this period. In the case of Boeing with a loss of 30%, in the case of Airbus with 14% and for Toyota with a loss of 15%.

2 A2C Algorithm Applied and Analysis

For each of the three shares, an Advantage Actor-Critic (A2C) agent was trained and evaluated against a simple buy-and-hold strategy. Per share, five independently trained agents were compared with the market. The tables below report the average returns of the trained agents, the average returns of the market (buy and hold), and the average percentage of cases in which the agent performed better than the market.

2.1 Boeing

Table 4: Boeing – average returns of agent vs. market.

	Agent 1	Agent 2	Agent 3	Agent 4	Agent 5
Trained agent	1.151250	0.982355	1.268870	1.211902	1.220214
Market (B&H)	1.220214	1.194680	1.411054	1.325877	1.139906
Agent better (%)	0.55	0.10	0.40	0.80	0.40

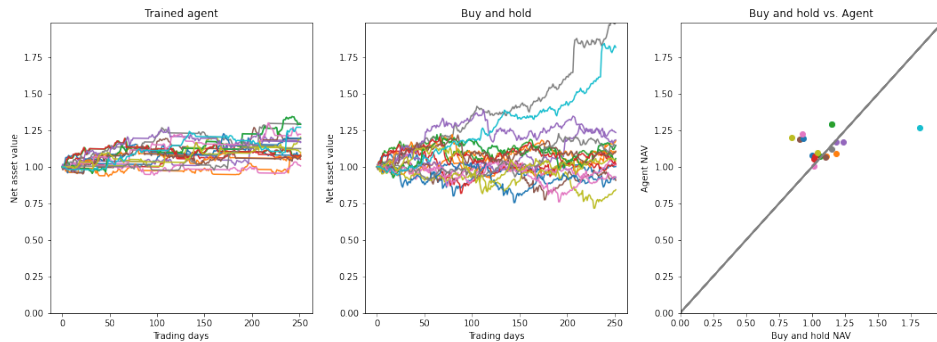


Figure 4: Boeing – cumulative return ratio: buy and hold vs. trained agent.

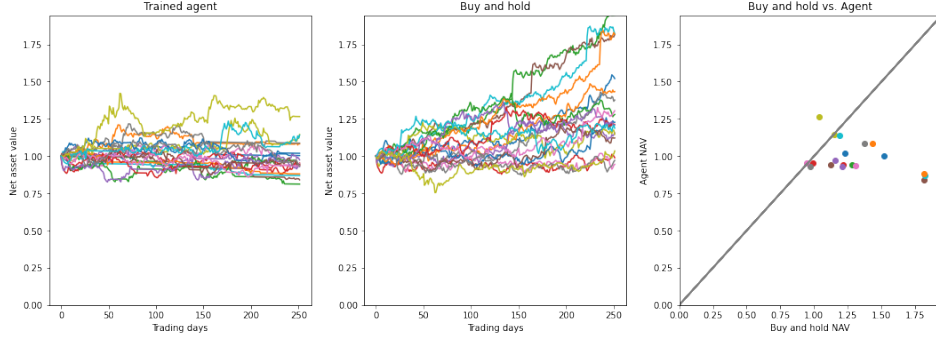


Figure 5: Boeing – actions chosen by the trained agents over time.

Commentary on the result for Boeing. Given those numbers, it can be stated that the A2C algorithm applied to the Boeing share would not result in an overall better performance than the simple buy-and-hold strategy. Only in 45% of the cases (on average) would the trained agent show a better performance. This is mainly due to one entrained agent which leads to better results than the market in only 10% of the cases. The analysis of the performed actions shows significantly different values, especially for the values for *short* and *cash*, distributed among the trained agents. It can therefore be assumed that the trained agent applied to this stock leads to inconsistent decisions and therefore to inconsistent and rather less promising results.

2.2 Airbus

Table 5: Airbus – average returns of agent vs. market.

	Agent 1	Agent 2	Agent 3	Agent 4	Agent 5
Trained agent	1.430267	1.570172	2.007212	1.276262	1.038152
Market (B&H)	1.217388	1.188968	1.168107	1.195885	1.237382
Agent better (%)	0.75	0.60	0.80	0.60	0.20

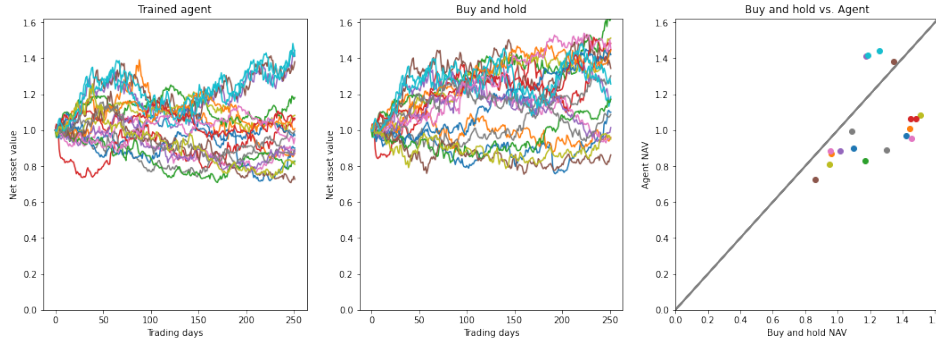


Figure 6: Airbus – cumulative return ratio: buy and hold vs. trained agent.

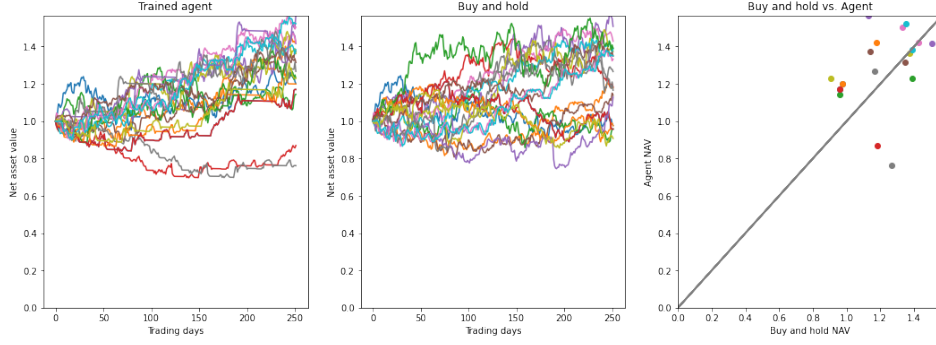


Figure 7: Airbus – actions chosen by the trained agents over time.

Commentary on the result for Airbus. Given those numbers, it can be stated that the A2C algorithm applied to the Airbus share would result in an overall better performance than the simple buy-and-hold strategy. In 60% of the cases (on average) the trained agent would show a better performance. The analysis of the performed actions shows similar values for four trained agents; only one trained agent differs a lot in terms of *cash*. It can therefore be assumed that the trained agent applied to this share could lead to consistent and promising results.

2.3 Toyota

Table 6: Toyota – average returns of agent vs. market.

	Agent 1	Agent 2	Agent 3	Agent 4	Agent 5
Trained agent	1.160707	1.442139	1.185587	1.263972	1.272148
Market (B&H)	0.994851	1.014057	1.044513	1.070050	1.003871
Agent better (%)	0.80	0.70	0.80	0.85	0.75

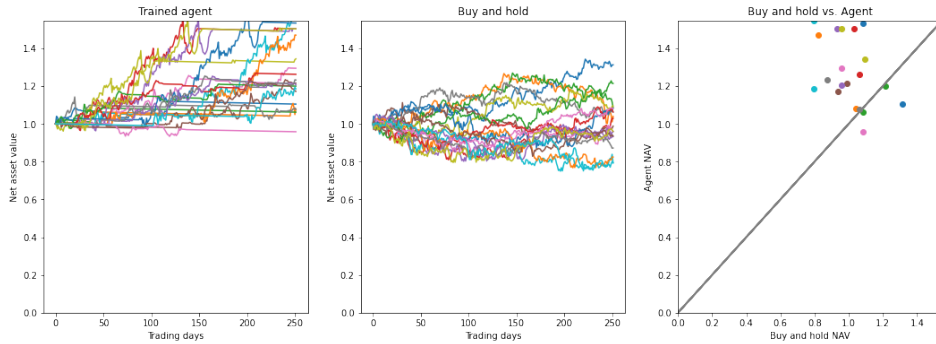


Figure 8: Toyota – cumulative return ratio: buy and hold vs. trained agent.

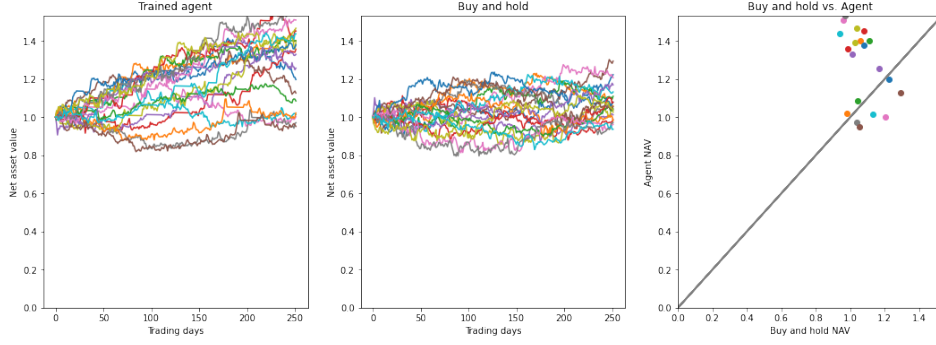


Figure 9: Toyota – actions chosen by the trained agents over time.

Commentary on the result for Toyota. Given those numbers, it can be stated that the A2C algorithm applied to the Toyota share would result in an overall better performance than the simple buy-and-hold strategy. In 78% of the cases (on average) the trained agent would show a better performance. The analysis of the performed actions shows different values for all trained agents. It can therefore be assumed that the trained agent applied to this share could lead to inconsistent behavior but could also lead to overall promising results.

3 Technical Indicators

3.1 Relative Strength Index (RSI)

The RSI indicator moves between zero and 100, plotting recent price gains versus recent price losses. The RSI levels therefore help in estimating momentum and trend strength:

$$RSI_t(n) = \frac{\sum_{k=0}^{n-1} (P_{t-i} - P_{t-i-1}) \mathbb{I}\{P_{t-i} > P_{t-i-1}\}}{|P_{t-1} - P_{t-i-1}|} \times 100, \quad (4)$$

where RSI_t is the Relative Strength Index at time t , P_t is the value of the index at time t , and n is the number of RSI periods. The RSI ranges from 0 to 100. A stock is considered *overbought* when its RSI is above 70, while it is regarded as *oversold* when the RSI is below 30.

3.2 Moving Average Convergence Divergence (MACD)

The MACD helps traders see the trend direction as well as the momentum of that trend. It also provides a number of trade signals. When the MACD is above zero, the price is in an upward phase; if the MACD is below zero, it has entered a bearish period. The indicator is composed of two lines: the MACD line and a signal line, which moves slower. When the MACD crosses below the signal line, it indicates that the price is falling; when the MACD line crosses above the signal line, the price is rising. The MACD is constructed based on moving averages: it is calculated by subtracting the longer exponential moving average (EMA) from the shorter EMA. The EMA is defined as

$$EMA_t = \frac{2}{n} \times (p_t - EMA_{t-1}) + EMA_{t-1}, \quad (5)$$

where EMA_t is the exponential moving average at time t and n is the number of periods for the EMA.

3.3 On-Balance Volume (OBV)

The on-balance volume indicator measures the positive and negative flow of volume over time. It is calculated using the closing price and the trading volume of a stock. Mathematically, the

OBV of a stock at time t is defined as

$$OBV_t = \begin{cases} OBV_{t-1} + V_t, & C_t > C_{t-1}, \\ OBV_{t-1} - V_t, & C_t < C_{t-1}, \end{cases} \quad (6)$$

where C_t and V_t are the closing price and the trading volume, respectively, at time t . By definition, the OBV increases when prices rise and vice versa.

3.4 Aroon Indicator

The Aroon indicator measures whether the price is hitting new highs or lows over the calculation period (typically 25). The formula for the Aroon oscillator is

$$Aroon\ Oscillator = Aroon\ Up - Aroon\ Down, \quad (7)$$

$$Aroon\ Up = 100 \times \frac{25 - \text{periods since 25-period high}}{25}, \quad (8)$$

$$Aroon\ Down = 100 \times \frac{25 - \text{periods since 25-period low}}{25}. \quad (9)$$

When the Aroon Up crosses above the Aroon Down, that is the first sign of a possible trend change. If the Aroon Up hits 100 and stays relatively close to that level while the Aroon Down stays near zero, that is positive confirmation of an uptrend.

3.5 Stochastic Oscillator

The stochastic oscillator measures the current price relative to the price range over a number of periods. Plotted between zero and 100, the idea is that when the trend is up, the price should be making new highs; in a downtrend, the price tends to make new lows. The stochastic tracks whether this is happening:

$$CL_t = P_t - \min(P_t, P_{t-1}, \dots, P_{t-s}), \quad (10)$$

$$HL_t = \max(P_t, P_{t-1}, \dots, P_{t-8}) - \min(P_t, P_{t-1}, \dots, P_{t-8}), \quad (11)$$

$$RSV = \frac{CL_t}{HL_t} \times 100, \quad (12)$$

$$K_t = \frac{2}{3}K_{t-1} + \frac{1}{3}RSV_t, \quad (13)$$

$$D_t = \frac{2}{3}D_{t-1} + \frac{1}{3}K_t, \quad (14)$$

where CL_t is measured as the lowest closing price in N recent days subtracted from the latest closing price; HL_t refers to the difference between the highest and the lowest closing price within N days; RSV_t is the CL_t over the HL_t ; the K value is the sum of 1/3 of the RSV value and 2/3 of the K value at lag one; and the D value is the sum of 1/3 of the K value and 2/3 of the D value at lag one. According to the above equations, the K and D values range from 0 to 100. Overbought signals are emitted as $K > 80$ and $K > D$; oversold signals are emitted as $K < 20$ and $K < D$.

3.6 Comments

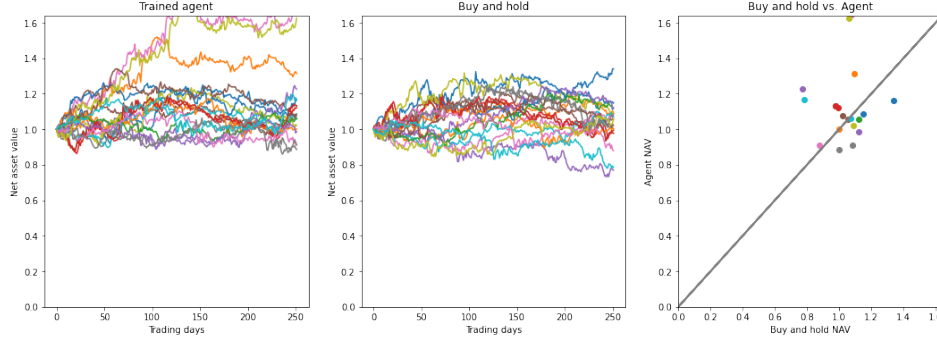


Figure 10: Selected technical indicators – comparison of revenues, buy and hold vs. agent.

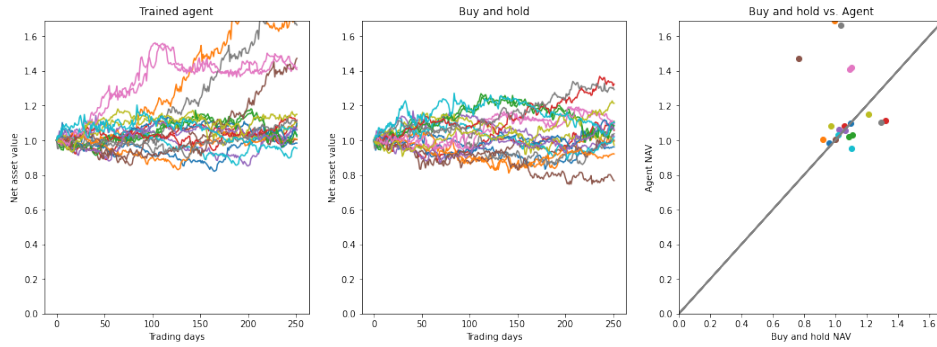


Figure 11: Selected technical indicators – actions chosen by the trained agents over time.

Comparing the graphs on the ratio of revenues between buy and hold and the trained agent, it is obvious that the revenues are similarly distributed, so there is no advantage for one of the strategies. However, the actions chosen over time give an insight into the fact that the trained agents suggest clearly different decisions. It can therefore be assumed that the trained agent applied to this share could lead to inconsistent behavior. Comparing those results with the agent trained with all indicators from above, it is well evident that in both described categories no better result can be achieved by only using the 5 indicators I selected.

4 Correlated State Variables

As an alternative to using all technical indicators, the 11 indicators that correlate most with tomorrow's log-return of the closing price were selected as state variables. This reduces the dimensionality of the state space while aiming to retain the most informative features.

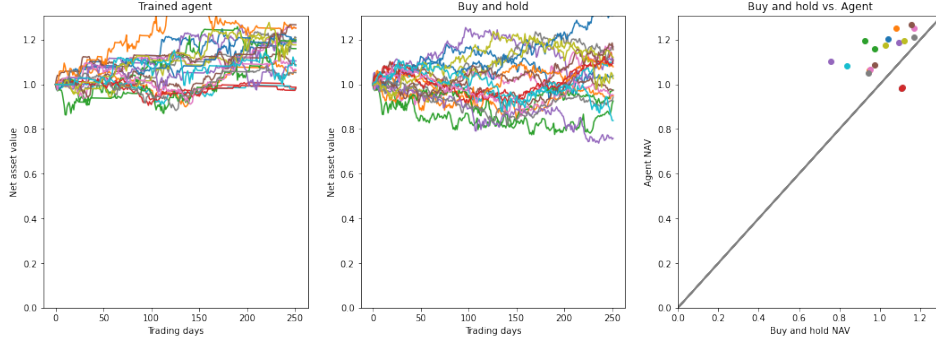


Figure 12: Correlated state variables – cumulative return ratio: buy and hold vs. agent.

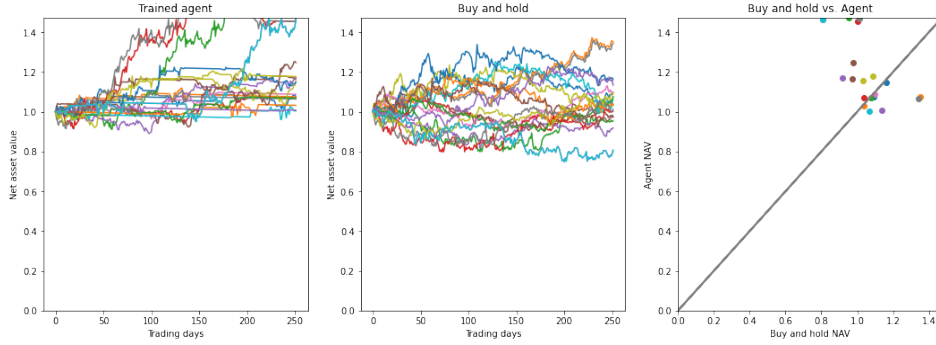


Figure 13: Correlated state variables – actions chosen by the trained agents over time.

Comments. Comparing the graphs on the ratio of revenues between buy and hold and the trained agent, it is obvious that the revenues are clearly better for the trained agent in comparison to buy and hold. However, the actions chosen over time give an insight into the fact that the trained agents suggest clearly different decisions. It can therefore be assumed that the trained agent applied to this share could lead to inconsistent behavior. Comparing those results with the agent trained with all indicators from above, it is evident that in the case of positive revenues, the trained agent using the 11 indicators that correlate most with tomorrow's log-return of the close price provides similar results as the one using all indicators.

5 Test Trained Agent on Different Time Periods

The data is divided into train sets and test sets according to the trading time. After the agents are trained over a specified timeframe, they are then evaluated on the test set consisting of a different timeframe.

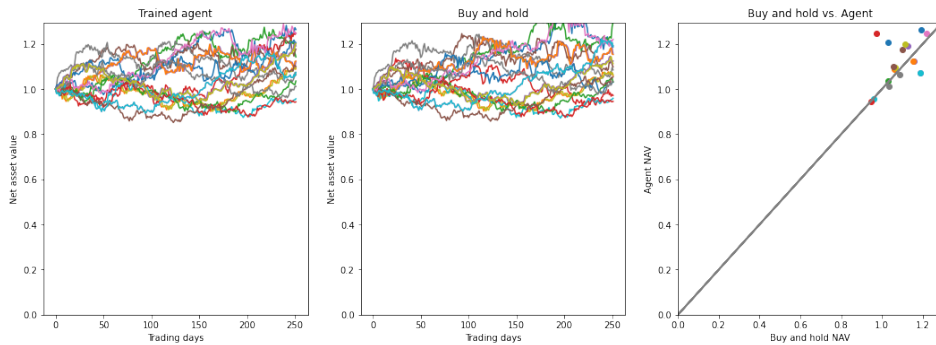


Figure 14: Train/test split – cumulative return ratio (split 1).

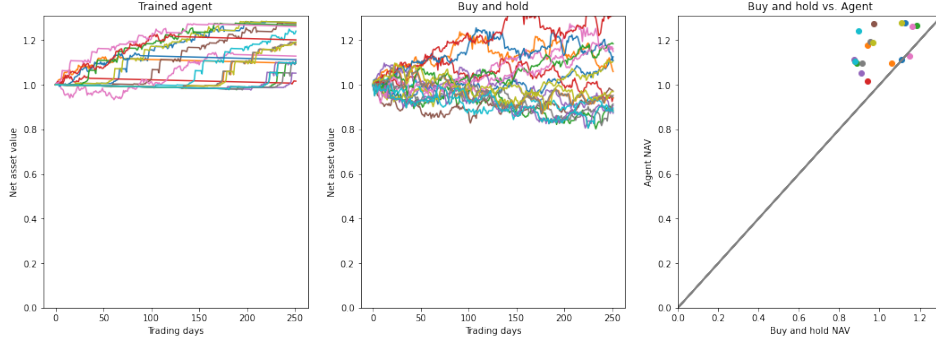


Figure 15: Train/test split – cumulative return ratio (split 2).

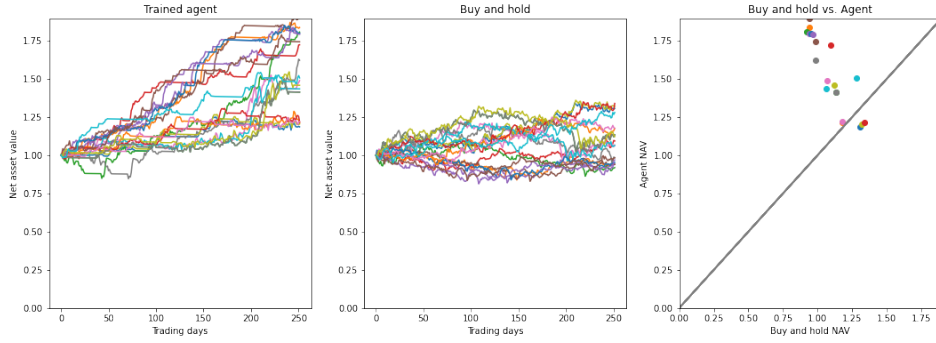


Figure 16: Train/test split – cumulative return ratio (split 4).

Comments. Results show that even when the trained agent shows promising results in terms of profits, if applied on the test set (except for one trained agent), positive rewards turn into negative rewards. Even if they show consistent behavior, the outcome isn't.

As a central conclusion, two things can be stated:

- If the agent is only trained over 4 years, no reliable predictions can be made about future behavior.
- The algorithm used here offers no possibility of predictability for future behavior in the field of stock trading and, on the contrary, performs worse than simple buy and hold.

6 Overall Findings and Limitations

The great advantage of RL is that an agent is able to learn and to perform actions in a dynamically changing environment. It thus allows the best possible decisions to be made during specific predefined points in time. The ability to train and improve the policy independently through several iteration steps therefore allows an optimal applicability in the field of algorithmic trading strategies. At the same time, the trading environment is characterized by more or less external influences such as the size and thus the power of larger investors, the influence of news and daily developments, decisions of individual investors, etc. It can further be assumed that the generalizability of an RL model is difficult to realize and it can therefore be a challenge to derive clear future behavioral implications.

Applied to my example, I could show that, for example, the application of the A2C algorithm to the Boeing stock over the last 10 years would lead to an average negative reward compared to a simple buy-and-hold strategy over the same period. This could be explained by the fact that especially drastic external influences have led the stock to behave in an unusual way. Out of 5 trained agents, one in particular showed very poor results of less than 1, which supports

my thesis of the time dependency and volatility of the model. Nevertheless, for the other two stocks of Airbus and Toyota over a time period of 10 years, the algorithm showed significantly better results in some cases, but without outputting consistent behavior once the agent is trained several times. It is further interesting to note that results in terms of positive rewards do not deviate much when all technical indicators are used on the one hand and only the 11 indicators that correlate most with tomorrow’s log-return of the close price on the other.

By training the agent on the Toyota stock and then testing the model over a different time period, I was able to show that the algorithm used on this stock is unlikely to generate promising results in the future.

7 Summary on One Selected Paper

7.1 Deep Robust Reinforcement Learning for Practical Algorithmic Trading

Yang Li, Wanshan Zheng and Zibin Zheng (2019)

In their 2019 paper, Yang Li, Wanshan Zheng and Zibin Zheng present a further development of previously used RL algorithms such as A3C and DQN for application in stock trading, a so-called *deep robust reinforcement learning* framework. To cope with the problem of uncontrollable external influences, called *noise* in their study, the following steps have to be performed first:

- Sampling random lengths of time units.
- To limit the influence of news, which is usually highest at opening time, the time points between 9:30 am to 11:30 am and 13:00 pm to 15:00 pm are omitted.
- Removal of the low volatility from the calculation in order to mitigate the effect caused by individual investors (as opposed to institutional investors).

In addition, to further reduce noise, the method of so-called SDAEs is utilized. The result of the last hidden layer is then fed into a LSTM layer in order allowing the agent to resolve the partial observability and discover latent patterns. For the model 5 stocks were chosen (3 NASDAQ, 1 S&P 500 stock index mini future, 1 HS300 stock index future). The results applied on their test set show that both algorithms output significantly better results in terms of annualized returns than both the original A3C and DQN algorithms as well as simple buy and hold. Furthermore, their study shows that the SDAEs-LSTM A3C algorithm performs significantly better than the SDAEs-LSTM DQN algorithm across all stocks.

Y. Li, W. Zheng and Z. Zheng, “Deep Robust Reinforcement Learning for Practical Algorithmic Trading,” in IEEE Access, vol. 7, pp. 108014–108022, 2019, doi: 10.1109/ACCESS.2019.2932789.

7.2 Related Work: Three Answers to the Generalization Problem

The three papers read for this report attack the same difficulty – the gap between in-sample fit and out-of-sample performance – from three different sides, and all three support the central observation of the 2026 update.

Representation. Li, Zheng and Zheng (2019) change the input side of the agent: stacked denoising autoencoders (SDAEs) clean the noisy market data and an LSTM layer resolves the partial observability of the market state. On ten years of minute data their extended DQN and A3C variants beat the plain algorithms and buy-and-hold in annualized return. Their evaluation, however, still rests on a single 90/10 train-test split, so the reported edge is an in-sample statement in the sense of our own comparison.

Environment. Kuo et al. (2021) change the training environment instead of the network. A LOB-GAN learns the ordering behaviour of the limit order book; its generator, combined with a matching engine, forms an interactive “Virtual Market” in which the portfolio agent is trained. Training in this simulated market improves out-of-sample portfolio performance by about four percent over other generalization strategies. This is the mirror image of our own result: a static environment built from historical prices is itself part of the generalization problem.

Policy. Niu, Li and Li (2022) keep the environment and diversify the policy set. MetaTrader first imitates several expert strategies (momentum, mean reversion, hindsight) to obtain a set of diverse base policies, and then trains a DQN meta-policy that recognises the current market regime and switches to the most suitable base policy. On DJIA, HSI and CSI100 it reports clearly better risk-adjusted returns than DeepTrader and the classical strategies.

Our own re-run (Chapter 8.2 and the 20-seed extension in the accompanying update report) sits between these three positions: the representation is unchanged (no SDAE), the environment is still built from historical prices, and the policy set is a single A2C agent. What was made robust instead is the evaluation itself – every configuration was repeated with twenty seeds. The outcome is sober and matches the papers’ implicit lesson: in-sample the agent wins clearly on all three stocks, but out-of-sample only Boeing keeps a significant edge over buy-and-hold, Airbus ends exactly on buy-and-hold and Toyota turns significantly worse. The in-sample margin does not predict the out-of-sample result. Read against the three papers, this suggests that robustness has to be built into the representation, the environment or the policy ensemble; a single-agent, single-environment setup like ours should not be expected to generalise on its own.

C.-H. Kuo, C.-T. Chen, S.-J. Lin and S.-H. Huang, “Improving Generalization in Reinforcement Learning-Based Trading by Using a Generative Adversarial Market Model,” in IEEE Access, vol. 9, 2021, doi: 10.1109/ACCESS.2021.3068269.

H. Niu, S. Li and J. Li, “MetaTrader: A Reinforcement Learning Approach Integrating Diverse Policies for Portfolio Optimization,” in Proc. 31st ACM Int. Conf. on Information and Knowledge Management (CIKM ’22), Atlanta, GA, USA, 2022, arXiv:2210.07771.

8 2026 Update: Fundamentals and a Re-run of the A2C Part

This chapter extends the original project with data up to 6 October 2026. It has two parts: an update of the fundamental figures (Section 8.1) and an independent re-run of the reinforcement-learning part on the new data window (Section 8.2). The re-run had to be carried out in a different software environment than the original notebook; every deviation is documented in Section 8.3 and should be read before comparing any number.

8.1 Updated Fundamental Figures

Methodology as in Chapter 1: P/E and P/S are trailing values, and the log-return is the sum of the daily log-returns of the adjusted close over twelve months.

Table 7: Fundamental figures: original notebook (2022) versus 6 October 2026.

Figure	Boeing (BA)	Airbus (AIR.PA)	Toyota (TM)
P/E 2022	n/a (KeyError)	19.887	10.375
P/E 2026	70.79	25.04	7.91
P/S 2022	1.502	1.458	0.0063
P/S 2026	1.60	1.93	0.68
Log-return 2022	−0.3025	−0.1451	−0.1486
Log-return 2026	−0.1558	−0.0945	−0.0743

P/E ratio. The statement from Section 1.2 is confirmed and partly sharpened. Airbus remains above the guideline value of 15 ($19.9 \rightarrow 25.0$), Toyota clearly below it ($10.4 \rightarrow 7.9$). Boeing delivers a value for the first time: **70.79**, which is an extremely high valuation multiple – a statement about valuation, not about the share price.

P/S ratio. The update uncovered a data error in the original notebook. The value of 0.0063 reported for Toyota (interpreted there as “close to zero”, i.e. very cheap) was a `yfinance` outlier. The actual value is **0.68**: Toyota is still the cheapest of the three, but the original statement “close to zero” is wrong. Boeing stands at 1.60 and Airbus at 1.93.

Log-returns. The core statement holds: all three shares are negative over the last twelve months as well (BA -15.6% , AIR -9.5% , TM -7.4%), so buy-and-hold would have lost money again. Over the longer horizon since the original data ended (August 2022), however, all three are clearly positive (BA $+20.5\%$, TM $+39.5\%$, AIR $+50$ to $+65\%$ over five years). The loss picture was therefore *window-specific*.

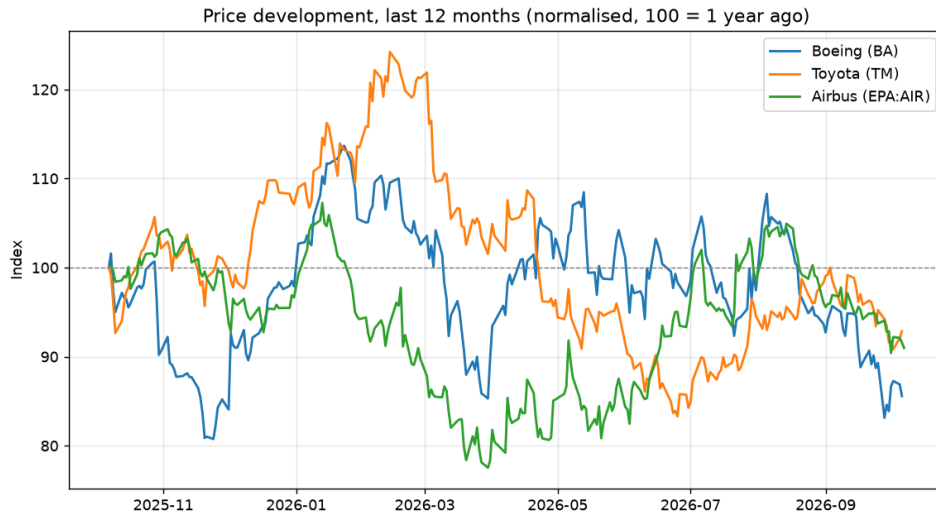


Figure 17: Price development of the three shares over the last twelve months (normalised, 100 = one year ago).

8.2 Re-run of the A2C Part on New Data

The protocol of the original notebook is reproduced: five agents with `A2C("MlpPolicy")`, 75,000 timesteps each, evaluated with `ai_trade_performance(num_plays=20)`. The test environment re-uses the feature list and the scaler of the training environment. Agent and buy-and-hold are evaluated on the same twenty episode seeds, so the comparison is fair.

Table 8: 2026 re-run on Boeing: final NAV per episode (start = 1.0), mean over twenty episodes per agent.

#	Seed	Train agent	Train B&H	Test agent	Test B&H	Test agent better
1	100	1.6705	0.9647	0.5616	0.9475	0 %
2	101	2.4073	0.9671	1.2403	0.9408	100 %
3	102	4.0534	0.9956	0.9828	0.9412	75 %
4	103	7.0580	0.9711	1.0106	0.9493	70 %
5	104	5.1547	0.9727	1.1326	0.9485	80 %
$\emptyset$		4.0688	0.9742	0.9856	0.9455	65 %

Training window 6 October 2021 to 31 May 2024 (in-sample), test window 1 June 2024 to 6 October 2026 (out-of-sample).

In-sample results are not evidence of skill. On the training window the agents reach an average NAV of 4.07 against 0.97 for buy-and-hold, and they beat the benchmark in every episode. This is not a result – the agents were trained on exactly this window, so the figure only shows that they memorised it.

Out-of-sample the picture is sober. On the test window the average NAV is 0.9856 against 0.9455 for buy-and-hold, with the agent ahead in **65 %** of the episodes – but with a very wide spread: agent 1 loses heavily (0.56), agent 2 wins clearly (1.24), and two of five agents lose against buy-and-hold in at least a quarter of the episodes. The advantage lies within seed variance and is **not significant**.

Buy-and-hold is below 1.0 in both windows, i.e. Boeing lost value over both periods. The question here is therefore “lose less”, not “win”. Overall the re-run **supports** the conclusion of the original notebook: A2C does not deliver a reliable outperformance.

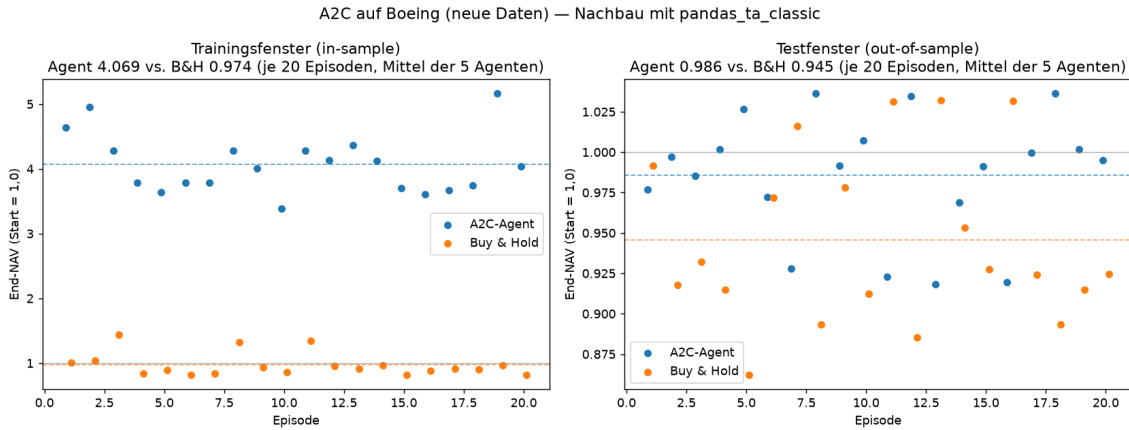


Figure 18: Final NAV per episode (mean of the five agents), left: training window (in-sample), right: test window (out-of-sample).

Table 9: Action distribution of the agents (one episode each).

#	Train short	Train cash	Train long	Test short	Test cash	Test long
1	268	18	348	239	17	299
2	295	39	300	304	24	227
3	297	24	313	279	29	247
4	246	212	176	260	201	94
5	296	15	323	309	12	234

All agents trade predominantly long; only agent 4 holds a large cash position. Short positions are almost always held, but rarely with a large weight.

8.3 Deviations from the Original Notebook

The re-run did not execute in the original conda environment (Python 3.8.8, stable-baselines3 1.6.0, gym 0.21, torch 1.12.1) but in a Linux container with a current stack. All substitutions are deliberate and documented.

Table 10: Substitutions in the 2026 re-run.

Aspect	Original notebook	2026 re-run
Price data	<code>yfinance</code> (live)	local CSV
Technical indicators	<code>pandas_ta</code> 0.3.14b0	<code>pandas_ta_classic</code>
Indicator count	204	348
Observation size	205	334
Parallelisation	multiprocessing pool	serial
RL stack	SB3 1.6.0 + gym 0.21	SB3 2.9 + gymnasium 1.4

The most important limitation is the observation size of 334 instead of 205. The agent sees more input features than in the original, so part of any performance difference may stem from that and not from the data update. In addition, `yfinance` is blocked from this host (HTTP 429 on every endpoint), which is why the price series is read from a local file, and the helper package `aif_course` is no longer available (the upstream repository returns HTTP 404). The environment had to be reconstructed locally; the exact substitutions are documented in the accompanying code.

8.4 Conclusion of the Update

The fundamental figures confirm the original assessment of Airbus (expensive) and Toyota (cheap), correct the erroneous Toyota P/S value, and add a very high P/E for Boeing. The loss picture of the log-returns is confirmed for the last twelve months, but turns out to be window-specific over a longer horizon. The A2C re-run on new data reproduces the original conclusion: on average the agent is slightly ahead of buy-and-hold out-of-sample, but the spread across seeds is so large that no reliable outperformance can be claimed.

9 A3C: A Second Algorithm, a Decomposition and a Generative Market Model

Where the question comes from. The original notebook and the re-run (Section 8.2) use *A2C*, i.e. synchronous learning. The obvious cross-check is *A3C*: several workers collect rollouts in parallel and push their gradients into a shared network (Hogwild style, only the optimiser step is locked). Environment, windows, episode seeds and evaluation functions are left unchanged, so the comparison measures the learning algorithm and not the surrounding machinery. The final evaluation again runs over **20 seeds (100–119)**. One peculiarity up front: A3C was built only *after* the A2C part was finished, so there is no notebook template against which the numbers could be mirrored.

9.1 The Baseline Has No Edge

Table 11: A3C on Boeing, 20 seeds. Margin = agent – buy-and-hold in NAV points (start = 1.0); p one-sided from the t -test “margin > 0”. “R/R” is the return/risk ratio in the test window.

Arm	Training	Test	95 % CI test	Hit rate test	p	R/R test
Baseline	+8.43	+0.019	[−0.06; +0.10]	51.0 %	0.32	−0.24
<code>ent_coef</code> 0.02	+8.37	+0.047	[−0.03; +0.12]	64.0 %	0.10	−0.37
<code>obs_noise</code> 0.01	+8.79	+0.038	[−0.03; +0.11]	57.2 %	0.14	−0.45
<code>nstep</code> 10	+6.71	+0.125	[+0.01; +0.24]	67.8 %	0.019	−0.05
Bundle (all three)	+7.69	+0.145	[+0.03; +0.26]	70.2 %	0.007	−0.03
Bundle + bootstrap	+0.02	+0.218	[+0.09; +0.35]	77.8 %	0.0012	+0.50
<code>nstep</code> 10 + bootstrap	+0.02	+0.218	[+0.09; +0.35]	72.5 %	0.0010	+0.38

The baseline repeats the A2C result: in the test window the margin is +0.019 NAV points, the agent wins in **51.0 %** of the episodes — a coin flip — and the one-sided p -value is 0.32. **An unmodified A3C agent has no demonstrable edge over buy-and-hold.** In the training window, at the same time, the margin is +8.43: the agent has memorised the one historical path.

9.2 Decomposition: Which Factor Carries the Effect?

The obvious move is a bundle of three changes: `nstep` 5 → 10 (n-step returns), `ent_coef` 0.01 → 0.02 (entropy bonus) and `obs_noise` 0 → 0.01 (Gaussian noise on the observation during training). The bundle lifts the test margin to +0.145 ($p = 0.007$). The only question is: *which* of the three factors does that?

Each factor was therefore run on its own, again with 20 seeds (rows 2–4 of Table 11). The result is unambiguous and was surprising to me:

`nstep` is the active ingredient. On its own, `nstep` 10 already reaches +0.125 ($p = 0.019$, hit rate 67.8 %) — that is **85 %** of the bundle’s effect. The paired test bundle against `nstep` alone gives +0.020 at $p = 0.81$: statistically **indistinguishable**. The bundle is therefore not a sum of levers but essentially `nstep` plus noise.

`ent_coef` and `obs_noise` are inert. On their own they yield +0.047 ($p = 0.64$) and +0.038 ($p = 0.72$) against the baseline, with 10 and 9 of 20 positive seed differences respectively — a coin flip. They do not act on the overfitting either: the in-sample margin stays at +8.37 and +8.79 against +8.43 for the baseline ($p = 0.90$ and 0.53). Only `nstep` 10 lowers it significantly to +6.71 ($p = 0.0015$) — and at the same time raises the test value.

Additivity. The sum of the three single deltas is +0.155, the bundle sits at +0.126; the residual is −0.029 against a spread of 0.61 ($p = 0.84$). The point estimate is thus slightly sub-additive (0.82×), but statistically indistinguishable from pure additivity — the noise dominates. There is no synergy in any case.

9.3 A Generative Market Model Instead of Rolling Windows

The second idea comes from the paper selection (Kuo et al., LOB-GAN, see Chapter 7.2): does the agent generalise better if it sees *several plausible market paths* instead of a single historical one? The obvious route would be rolling training windows — and that is **not buildable** in this protocol. The environment forces more than $2 \cdot 252 = 504$ observations per environment (`assert len(df) > 2*trading_days`), and `trading_days` simultaneously sets the episode length of 252

steps; the minimum therefore cannot be lowered without breaking the protocol. The training window, however, has only **634** usable rows, and the price series starts exactly at the window’s beginning. Two genuine sub-windows would each need around 550 rows — impossible. The code now checks this itself and aborts with a clear message.

Instead, a **block bootstrap** generates synthetic price series. What matters is *what* is resampled: price levels would be wrong, because every block seam would create an artificial jump (a block ends at 100, the next starts at 80 — minus 20% overnight). The *daily shape* is therefore bootstrapped: $\log(\text{Close}_t/\text{Close}_{t-1})$, $\log(\text{Open}_t/\text{Close}_{t-1})$, $\log(\text{High}_t/\text{Close}_t)$, $\log(\text{Low}_t/\text{Close}_t)$ and $\log(\text{Volume}_t)$. Blocks of 20 consecutive days are drawn with replacement and the path is rebuilt from the first real close. Within the blocks the autocorrelation is preserved, the curve is continuous by construction, and the agent never sees the *real* training path. Cross-check: the spread of the daily returns is 0.0229 and 0.0247 in the synthetic series against 0.0239 in the real series.

9.4 Isolation: the Bootstrap Carries, the Two Passengers Do Not

In the “bundle + bootstrap” row the bootstrap still sits together with the two inert factors. The isolating run `nstep 10 + bootstrap` without `ent_coef` and `obs_noise` closes that gap (last row):

Table 12: Paired tests over the 20 seed differences (df= 19), test margin.

Comparison	Δ test margin	t	p
<code>nstep 10</code> – baseline	+0.107	1.69	0.107
<code>ent_coef 0.02</code> – baseline	+0.028	0.47	0.643
<code>obs_noise 0.01</code> – baseline	+0.020	0.36	0.720
bundle – baseline	+0.126	1.98	0.062
bundle – <code>nstep 10</code>	+0.020	0.24	0.810
<code>nstep 10 + bootstrap</code> – baseline	+0.200	2.35	0.030
<code>nstep 10 + bootstrap</code> – <code>nstep 10</code>	+0.093	1.16	0.260
<code>nstep 10 + bootstrap</code> – bundle + bootstrap	+0.0002	0.003	0.998

The bootstrap effect survives in isolation — and was never carried by the passengers.

The isolated increment is +0.093 and thus *exactly* the same value as the contaminated estimate (+0.093); the ratio is 1.00. The effect has therefore neither shrunk nor disappeared. One caveat: on its own it is **not significant** ($p = 0.26$, 12 of 20 seeds positive, median +0.138, trimmed mean +0.112) — the point estimate is stable, the sample is too small.

`nstep 10 + bootstrap` is indistinguishable from “bundle + bootstrap” (+0.0002, $p = 0.998$). `ent_coef` and `obs_noise` are thus **finally proven to be passengers**: the two runs differ only in those two parameters and produce practically the same distribution. (Incidentally a consistency check that the bootstrap replicates are deterministic.)

A significant edge for the first time. `nstep 10 + bootstrap` is significantly better than the baseline (+0.200, $p = 0.030$) and its own test margin is clearly different from zero ($p = 0.0010$, two-sided 0.0019). This is the *first* arm in this project to break the 5% threshold in the paired test against the baseline — neither `nstep` alone ($p = 0.107$) nor the bundle ($p = 0.062$) manages that. The result is a single-stock one, however: Section 9.6 repeats the comparison on the other two stocks and shows that the advantage does *not* transfer.

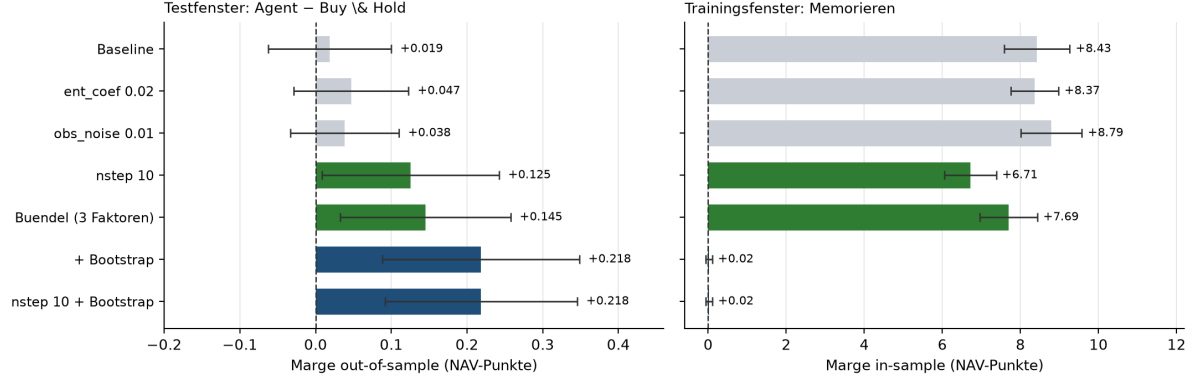


Figure 19: One-factor decomposition and block bootstrap, 20 seeds. Left the test margin (error bars: 95 % CI), right the margin in the training window. The two bootstrap arms (dark) have practically zero in-sample edge and the largest out-of-sample one.

9.5 What the Overfitting Picture Shows

Figure 19 makes the core visible. The four arms that train on the real window carry an in-sample margin of between +6.7 and +8.8 NAV points — and almost none of it survives out-of-sample. The two bootstrap arms have an in-sample margin of +0.02, i.e. *zero* edge, and at the same time deliver the best test values. They never saw the historical path; that they do not win in the training window is therefore not a weakness but the mechanism. An in-sample edge of +8 is thus exposed as what it is: memorisation.

9.6 Robustness Check: the Other Two Stocks

The decomposition above was derived on a single stock, so the natural question is whether it is a property of the method or of Boeing. The whole protocol — same environment, windows, episode seeds and evaluation, 20 seeds — was therefore repeated for **Airbus** and **Toyota**, each with the three arms baseline, **nstep 10** and **nstep 10 + bootstrap**.

Table 13: Replication on all three stocks, 20 seeds. Test margin = agent – buy-and-hold (NAV points); the two right-hand columns are the paired difference against the baseline.

Stock	baseline	nstep 10	nstep 10 + boot	Δ nstep 10	Δ + boot
Boeing	+0.019	+0.125	+0.218	+0.107 ($p = 0.11$)	+0.200 ($p = 0.030$)
Airbus	−0.035	+0.071	−0.180	+0.105 ($p = 0.11$)	−0.145 ($p = 0.012$)
Toyota	−0.184	−0.188	−0.009	−0.004 ($p = 0.94$)	+0.175 ($p = 0.008$)
pooled AIR+TM	—	—	—	+0.051 ($p = 0.24$)	+0.015 ($p = 0.75$)

Table 13 and Figure 20 give the sober answer: the *mechanism* replicates, the *benefit* does not.

The mechanism is reproducible. On every stock the in-sample margin of the two bootstrap-free arms is large (+2.8/+2.0 on Airbus, +5.0/+3.9 on Toyota) while the bootstrap arms sit at ≈ 0 (−0.26 Airbus, −0.40 Toyota). The bootstrap removes memorisation everywhere, exactly as on Boeing.

The out-of-sample benefit is not. The paired difference `nstep 10 + bootstrap` – baseline is $+0.200$ on Boeing ($p = 0.030$) and $+0.175$ on Toyota ($p = 0.008$) — but -0.145 on Airbus ($p = 0.012$): there the bootstrap makes the agent significantly *worse*. Pooled over Airbus and Toyota the effect is $+0.015$ ($p = 0.75$, $n = 40$) — i.e. no effect. The single-factor lever `nstep 10` is equally unstable: $+0.107$ (Boeing) and $+0.105$ (Airbus) are almost identical, Toyota is exactly zero (-0.004 , $p = 0.94$), and the pooled value ($+0.051$, $p = 0.24$) is not significant.

Consequence. The “first significant edge” of the previous subsection is a **single-stock result** and must not be read as a general property of the method. With three stocks, one significant improvement and one significant deterioration is exactly the pattern expected from noise plus a stock-specific effect; the pooled test — the honest summary — finds nothing. The defensible statement is therefore narrower and, for a course project, more instructive: *the mechanism is reproducible, the payoff is not*. This also matches the lesson of the three selected papers (Chapter 7.2): generalisation has to be built in, and a single lever that helps on one price series should not be expected to transfer to another.

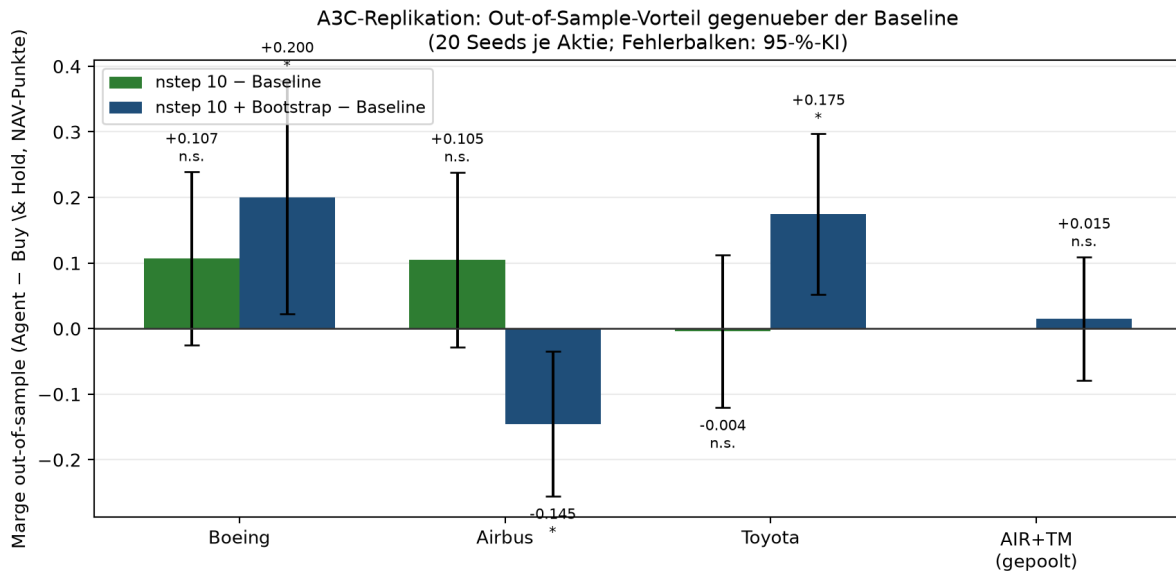


Figure 20: Out-of-sample margin of the two levers against the baseline, per stock and pooled (Airbus+Toyota, $n = 40$). Error bars: 95 % CI of the paired difference; * marks $p < 0.05$. Boeing and Toyota improve with the bootstrap, Airbus deteriorates; pooled the effect vanishes.

9.7 Generalization Check: Six Further Stocks

The three stocks above are not a neutral sample: they are the ones the course picked, and two of them (Boeing, Airbus) sit in the same industry. If the bootstrap effect is a property of the *method*, it has to show up on other price series as well; if it is stock-specific, six further, deliberately *sector-diverse* series should scatter around zero. The protocol was therefore repeated unchanged — same environment, windows, episode seeds and evaluation, 20 seeds, the three arms baseline, `nstep 10` and `nstep 10 + bootstrap` — on **LMT** (defence), **CAT** (industrial), **JPM** (banking), **KO** (consumer staples), **NVDA** (technology) and **XOM** (energy).

Table 14: Generalization check on six further stocks, 20 seeds each. Test margin = agent – buy-and-hold (NAV points); the right-hand column is the paired difference against the baseline, p two-sided from the t -test. The two pooled rows use the seed differences of the six new stocks ($n = 120$) and of all nine price series ($n = 180$, i.e. including Boeing, Airbus and Toyota).

Stock	baseline	nstep 10	nstep 10 + boot	Δ + boot – baseline
LMT	−0.177	−0.213	−0.165	+0.012 ($p = 0.87$)
CAT	−0.730	−0.813	−0.777	−0.047 ($p = 0.79$)
JPM	−0.153	−0.190	−0.221	−0.067 ($p = 0.099$)
KO	−0.308	−0.270	−0.329	−0.021 ($p = 0.78$)
NVDA	−0.089	−0.058	−0.337	−0.248 ($p = 0.012$)
XOM	−0.700	−0.689	−0.544	+0.157 ($p = 0.004$)
pooled, six new ($n = 120$)	—	—	—	−0.036 ($p = 0.36$)
pooled, all nine ($n = 180$)	—	—	—	+0.002 ($p = 0.96$)

The mechanism replicates for a third time. On all six new stocks the bootstrap-free arms carry a large in-sample margin (+1.4 to +30.2 NAV points) while the bootstrap arm sits at zero or below it: +0.05 (LMT), −0.31 (CAT), −0.29 (JPM), −0.01 (KO), −1.74 (NVDA) and +0.24 (XOM). That the block bootstrap removes the memorisation of the historical path is thus reproducible across nine price series — this part is a stable property of the procedure.

The out-of-sample benefit is not. Four of the six new stocks move only within noise (LMT +0.012, $p = 0.87$; CAT −0.047, $p = 0.79$; JPM −0.067, $p = 0.099$; KO −0.021, $p = 0.78$), and the two significant ones point in *opposite* directions: **XOM** improves by +0.157 ($p = 0.004$) while **NVDA** deteriorates by −0.248 ($p = 0.012$). Pooled over the six the effect is −0.036 ($p = 0.36$, $n = 120$), over all nine price series +0.002 ($p = 0.96$, $n = 180$). Neither is distinguishable from zero.

A side observation. On most of the new stocks the baseline agent itself loses to buy-and-hold in the test window (CAT −0.73, XOM −0.70, KO −0.31): on strongly trending series the conservative buy-and-hold is hard to beat, and the arms then differ only in how much they lose.

Consequence. The generalization hypothesis is **not confirmed**. The nstep 10 + bootstrap advantage seen on Boeing and Toyota (Section 9.6) does not reproduce on six further, sector-diverse stocks; across nine price series in total, two significant improvements and two significant deteriorations cancel out. The honest summary of the whole A3C part is thus: the *mechanism* — a bootstrap that removes memorisation — is robust, its *payoff* is not. A lever that helps on one price series must not be expected to transfer to another; that is precisely the lesson of the three selected papers (Chapter 7.2).

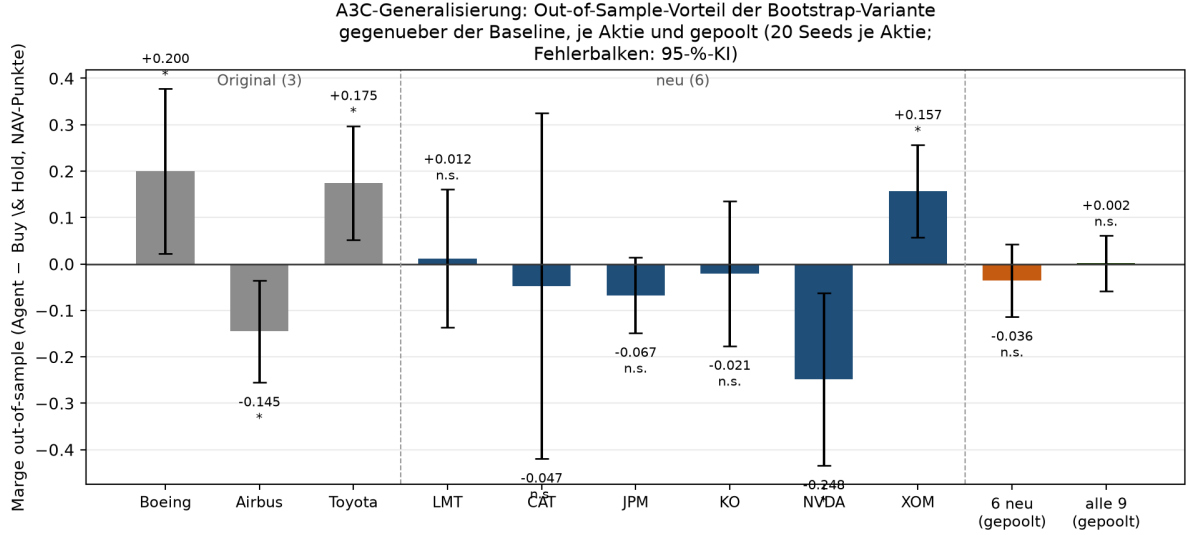


Figure 21: Out-of-sample advantage of the bootstrap variant over the baseline on nine price series, per stock and pooled. Error bars: 95 % CI of the paired difference; * marks $p < 0.05$. The three original stocks scatter widely; of the six new ones two are significant in opposite directions and four stay within noise. Pooled, the effect vanishes.

9.8 Limitations

20 seeds are a small sample. The confidence intervals in Table 11 are wide, and the central bootstrap increment is underpowered at $p = 0.26$. What is robust: that the baseline has no edge, that `nstep 10` explains the bundle, that the two remaining factors do nothing, and that `nstep 10 + bootstrap` beats the baseline significantly. What is not robust is the claim that the bootstrap addition is significant *by itself*.

Only five different synthetic paths. With 20 agents and five bootstrap replicates, groups of four seeds share the same synthetic series. The seeds then vary only the exploration, not the environment. A run with one path per agent would be the next clean step.

The in-sample margin of the bootstrap arms is not comparable. It is measured on the real window, on which these agents never trained; the row is therefore readable only as an overfitting indicator, not as a performance measure.

Data and environment. The limitations from Section 8.3 apply: observation vector 334 instead of 205, local CSVs instead of `yfinance`. A3C uses the same environment as A2C, so the deviations affect both arms equally.

9.9 Conclusion of the A3C Part

The baseline confirms the A2C finding on a second algorithm: no demonstrable edge over buy-and-hold. The decomposition shows that the effect of the obvious “regularisation” actually comes from the **credit assignment** (`nstep 5` $\rightarrow$ `10`); entropy bonus and observation noise are ineffective at this magnitude. The **block bootstrap** delivers, as a second lever, the best test value and — on Boeing — the first significant edge over the baseline, but only in combination. The robustness check on the other two stocks (Section 9.6) shows that this edge does not transfer: the mechanism is reproducible, the payoff is not. The generalization check on six further, sector-diverse stocks (Section 9.7) confirms this: pooled over all nine price series the bootstrap advantage is $+0.002$ ($p = 0.96$, $n = 180$). Conceptually this is the more interesting finding: the

agent does not need the real historical path to be level with or better than buy-and-hold on the test window; many plausible paths are enough.

A Notebook Code

The complete Python code of the project notebook is listed below. It contains all data acquisition, feature engineering, environment setup, training and evaluation steps in the order in which they were executed in the Jupyter notebook.

```
1 # ---- Cell 3 (run 1) ----
2 !pip install pandas-ta
3 !pip install stable-baselines3
4 !pip install sklearn
5
6 # ---- Cell 4 (run 2) ----
7 import aif_environment
8 import yfinance as yf
9 import pandas as pd
10 import numpy as np
11
12 # ---- Cell 6 (run 3) ----
13 boeing = yf.Ticker('BA')
14 ba_df = boeing.history(period="10y")
15 ba_df['Close'].plot(title="BA's stock price")
16
17 # ---- Cell 8 (run 4) ----
18 airbus = yf.Ticker('AIR.PA')
19 air_df = airbus.history(period="10y")
20 air_df['Close'].plot(title="AIR's stock price")
21
22 # ---- Cell 10 (run 5) ----
23 toyota = yf.Ticker('TM')
24 tm_df = toyota.history(period="10y")
25 tm_df['Close'].plot(title="TM's stock price")
26
27 # ---- Cell 13 (run 6) ----
28 print((boeing.info.get('trailingPE')))
29
30 # ---- Cell 15 (run 7) ----
31 print((airbus.info.get('trailingPE')))
32
33 # ---- Cell 17 (run 8) ----
34 print((toyota.info.get('trailingPE')))
35
36 # ---- Cell 20 (run 9) ----
37 print((boeing.info['priceToSalesTrailing12Months']))
38
39 # ---- Cell 22 (run 10) ----
40 print((airbus.info['priceToSalesTrailing12Months']))
41
42 # ---- Cell 24 (run 11) ----
43 print((toyota.info['priceToSalesTrailing12Months']))
44
45 # ---- Cell 28 (run 12) ----
46 bo = yf.download('BA', start = '2021-09-01', end='2022-08-31')
47 bo.reset_index(inplace=True)
48
49 # ---- Cell 29 (run 13) ----
50 bo['Log_Returns'] = np.log(bo['Adj Close']) - np.log(bo['Adj Close'].shift(1))
51 bo.head()
52
53 # ---- Cell 30 (run 14) ----
```

```

54 bo['Log>Returns'].sum()
55
56 # ---- Cell 32 (run 15) ----
57 ai = yf.download('AIR.PA', start = '2021-09-01', end='2022-08-31')
58 ai.reset_index(inplace=True)
59
60 # ---- Cell 33 (run 16) ----
61 ai['Log>Returns'] = np.log(ai['Adj Close']) - np.log(ai['Adj Close'].shift(1))
62 ai.head()
63
64 # ---- Cell 34 (run 17) ----
65 ai['Log>Returns'].sum()
66
67 # ---- Cell 36 (run 18) ----
68 tm = yf.download('TM', start = '2021-09-01', end='2022-08-31')
69 tm.reset_index(inplace=True)
70
71 # ---- Cell 37 (run 19) ----
72 tm['Log>Returns'] = np.log(tm['Adj Close']) - np.log(tm['Adj Close'].shift(1))
73 tm.head()
74
75 # ---- Cell 38 (run 20) ----
76 tm['Log>Returns'].sum()
77
78 # ---- Cell 41 (run 21) ----
79 from stable_baselines3 import A2C
80
81 # load the trained agents
82 model5 = A2C.load('aif_course-master/A2C_showcase_agent5')
83 model4 = A2C.load('aif_course-master/A2C_showcase_agent4')
84 model3 = A2C.load('aif_course-master/A2C_showcase_agent3')
85 model2 = A2C.load('aif_course-master/A2C_showcase_agent2')
86 model = A2C.load('aif_course-master/A2C_showcase_agent1')
87
88 # ---- Cell 42 (run 22) ----
89 from aif_environment import TradingEnvironment
90 from aif_analysis import ai_trade_performance
91
92 # ---- Cell 43 (run 23) ----
93 env = TradingEnvironment(ticker = 'BA', start_date = '2012-01-01', end_date = '2017-12-31')
94
95 # and examine the agent's performance
96 res = ai_trade_performance(env, model5, num_plays = 20)
97 res
98
99 # ---- Cell 44 (run 24) ----
100 res = ai_trade_performance(env, model4, num_plays = 20)
101 res
102
103 # ---- Cell 45 (run 25) ----
104 res = ai_trade_performance(env, model3, num_plays = 20)
105 res
106
107 # ---- Cell 46 (run 26) ----
108 res = ai_trade_performance(env, model2, num_plays = 20)
109 res
110
111 # ---- Cell 47 (run 27) ----
112 res = ai_trade_performance(env, model, num_plays = 20)
113 res
114
115 # ---- Cell 48 (run 28) ----
116 from aif_analysis import analyze_actions_taken

```

```

117 analyze_actions_taken(env, model, plot_actions = True)
118
119 # ---- Cell 49 (run 29) ----
120 analyze_actions_taken(env, model2, plot_actions = True)
121
122 # ---- Cell 50 (run 30) ----
123 analyze_actions_taken(env, model3, plot_actions = True)
124
125 # ---- Cell 51 (run 31) ----
126 analyze_actions_taken(env, model4, plot_actions = True)
127
128 # ---- Cell 52 (run 32) ----
129 analyze_actions_taken(env, model5, plot_actions = True)
130
131 # ---- Cell 54 (run 33) ----
132 returns_ta = np.array ([[1.151250], [0.982355], [1.268870], [1.211902], [1.220214]])
133 returns_bh = np.array ([[1.248442], [1.194680], [1.411054], [1.325877], [1.139906]])
134 agent_better = ([0.55], [0.1], [0.4], [0.8], [0.4])
135
136 average_return_ta = np.mean(returns_ta)
137 print("The average return of all trained agents is:", average_return_ta)
138 average_return_bh = np.mean(returns_bh)
139 print("The average return of the market over all 5 iterations is:", average_return_bh)
140 agent_better_average = np.mean(agent_better)
141 print("The average in which the agent performs better than the market:", agent_better_average)
142
143 # ---- Cell 57 (run 34) ----
144 model5_AIR = A2C.load('aif_course-master/A2C_showcase_agent5_AIR')
145 model4_AIR = A2C.load('aif_course-master/A2C_showcase_agent4_AIR')
146 model3_AIR = A2C.load('aif_course-master/A2C_showcase_agent3_AIR')
147 model2_AIR = A2C.load('aif_course-master/A2C_showcase_agent2_AIR')
148 model_AIR = A2C.load('aif_course-master/A2C_showcase_agent1_AIR')
149
150 # ---- Cell 58 (run 35) ----
151 env_AIR = TradingEnvironment(ticker = 'AIR.PA', start_date = '2012-01-01', end_date = '
152     2017-12-31')
153
154 # and examine the agent's performance
155 res = ai_trade_performance(env_AIR, model_AIR, num_plays = 20)
156 res
157
158 # ---- Cell 59 (run 36) ----
159 res = ai_trade_performance(env_AIR, model2_AIR, num_plays = 20)
160 res
161
162 # ---- Cell 60 (run 37) ----
163 res = ai_trade_performance(env_AIR, model3_AIR, num_plays = 20)
164 res
165
166 # ---- Cell 61 (run 38) ----
167 res = ai_trade_performance(env_AIR, model4_AIR, num_plays = 20)
168 res
169
170 # ---- Cell 62 (run 39) ----
171 res = ai_trade_performance(env_AIR, model5_AIR, num_plays = 20)
172 res
173
174 # ---- Cell 63 (run 40) ----
175 from aif_analysis import analyze_actions_taken
176
177 analyze_actions_taken(env_AIR, model_AIR, plot_actions = True)
178

```

```

179 # ---- Cell 64 (run 41) ----
180 analyze_actions_taken(env_AIR, model2_AIR, plot_actions = True)
181
182 # ---- Cell 65 (run 42) ----
183 analyze_actions_taken(env_AIR, model3_AIR, plot_actions = True)
184
185 # ---- Cell 66 (run 43) ----
186 analyze_actions_taken(env_AIR, model4_AIR, plot_actions = True)
187
188 # ---- Cell 67 (run 44) ----
189 analyze_actions_taken(env_AIR, model5_AIR, plot_actions = True)
190
191 # ---- Cell 69 (run 45) ----
192 returns_ta = np.array ([[1.430267], [1.570172], [2.007212], [1.276262], [1.038152]])
193 returns_bh = np.array ([[1.217388], [1.188968], [1.168107], [1.195885], [1.237382]])
194 agent_better = ([0.75], [0.60], [0.80], [0.60], [0.20])
195
196 average_return_ta = np.mean(returns_ta)
197 print("The average return of all trained agents is:", average_return_ta)
198 average_return_bh = np.mean(returns_bh)
199 print("The average return of the market over all 5 iterations is:", average_return_bh)
200 agent_better_average = np.mean(agent_better)
201 print("The average in which the agent performs better than the market:", agent_better_average)
202
203 # ---- Cell 72 (run 46) ----
204
205 # load the trained agents
206 model5_TM = A2C.load('aif_course-master/A2C_showcase_agent5_TM')
207 model4_TM = A2C.load('aif_course-master/A2C_showcase_agent4_TM')
208 model3_TM = A2C.load('aif_course-master/A2C_showcase_agent3_TM')
209 model2_TM = A2C.load('aif_course-master/A2C_showcase_agent2_TM')
210 model_TM = A2C.load('aif_course-master/A2C_showcase_agent1_TM')
211
212 # ---- Cell 73 (run 47) ----
213 env_TM = TradingEnvironment(ticker = 'TM', start_date = '2012-01-01', end_date = '2017-12-31')
214
215 # and examine the agent's performance
216 res = ai_trade_performance(env_TM, model_TM, num_plays = 20)
217 res
218
219 # ---- Cell 74 (run 48) ----
220 res = ai_trade_performance(env_TM, model2_TM, num_plays = 20)
221 res
222
223 # ---- Cell 75 (run 49) ----
224 res = ai_trade_performance(env_TM, model3_TM, num_plays = 20)
225 res
226
227 # ---- Cell 76 (run 50) ----
228 res = ai_trade_performance(env_TM, model4_TM, num_plays = 20)
229 res
230
231 # ---- Cell 77 (run 51) ----
232 res = ai_trade_performance(env_TM, model5_TM, num_plays = 20)
233 res
234
235 # ---- Cell 78 (run 52) ----
236 analyze_actions_taken(env_TM, model_TM, plot_actions = True)
237
238 # ---- Cell 79 (run 53) ----
239 analyze_actions_taken(env_TM, model2_TM, plot_actions = True)
240
241 # ---- Cell 80 (run 54) ----

```

```

242 analyze_actions_taken(env_TM, model3_TM, plot_actions = True)
243
244 # ---- Cell 81 (run 55) ----
245 analyze_actions_taken(env_TM, model4_TM, plot_actions = True)
246
247 # ---- Cell 82 (run 56) ----
248 analyze_actions_taken(env_TM, model5_TM, plot_actions = True)
249
250 # ---- Cell 84 (run 57) ----
251 returns_ta = np.array ([[1.160707], [1.442139], [1.185587], [1.263972], [1.272148]])
252 returns_bh = np.array ([[0.994851], [1.014057], [1.044513], [1.07005], [1.003871]])
253 agent_better = ([0.80], [0.70], [0.80], [0.85], [0.75])
254
255 average_return_ta = np.mean(returns_ta)
256 print("The average return of all trained agents is:", average_return_ta)
257 average_return_bh = np.mean(returns_bh)
258 print("The average return of the market over all 5 iterations is:", average_return_bh)
259 agent_better_average = np.mean(agent_better)
260 print("The average in which the agent performs better than the market:", agent_better_average)
261
262 # ---- Cell 100 (run 58) ----
263 import yfinance as yf
264 import pandas as pd
265
266 stock = yf.Ticker("TM")
267
268 df = stock.history(start = "2012-01-01", end = "2017-12-31")
269 df.ta.strategy()
270 df["STOCHd_14_3_3"]
271
272 # ---- Cell 101 (run 59) ----
273 from aif_environment import TradingEnvironment
274
275 my_indicators = ["AROONOSC_14", "RSI_14", "MACD_12_26_9", "OBV", "STOCHk_14_3_3", "STOCHd_14_3_3"]
276
277 env_ti = TradingEnvironment(ticker = "TM", start_date = '2012-01-01', end_date = '2017-12-31',
278                             use_variables = my_indicators)
279
280 # ---- Cell 102 (run 60) ----
281 from stable_baselines3 import A2C
282
283 # ---- Cell 103 (run 61) ----
284 model_ti5 = A2C('MlpPolicy', env_ti, verbose = 1)
285 model_ti5.learn(total_timesteps = 75000)
286
287 # ---- Cell 104 (run 62) ----
288 model_ti4 = A2C('MlpPolicy', env_ti, verbose = 0)
289 model_ti4.learn(total_timesteps = 75000)
290
291 # ---- Cell 105 (run 63) ----
292 model_ti3 = A2C('MlpPolicy', env_ti, verbose = 0)
293 model_ti3.learn(total_timesteps = 75000)
294
295 # ---- Cell 106 (run 64) ----
296 model_ti2 = A2C('MlpPolicy', env_ti, verbose = 0)
297 model_ti2.learn(total_timesteps = 75000)
298
299 # ---- Cell 107 (run 65) ----
300 model_ti = A2C('MlpPolicy', env_ti, verbose = 0)
301 model_ti.learn(total_timesteps = 75000)
302
303 # ---- Cell 108 (run 66) ----

```

```

303 overall, fnavs, mnavs = ai_trade_performance(env_ti, model_ti, num_plays = 20)
304 overall, fnavs, mnavs
305 actions_ti = analyze_actions_taken(env_ti, model_ti, plot_actions = True)
306
307 # ---- Cell 109 (run 67) ----
308 overall, fnavs, mnavs = ai_trade_performance(env_ti, model_ti2)
309 actions_ti = analyze_actions_taken(env_ti, model_ti2, plot_actions = True)
310
311 # ---- Cell 110 (run 68) ----
312 overall, fnavs, mnavs = ai_trade_performance(env_ti, model_ti3)
313 actions_ti = analyze_actions_taken(env_ti, model_ti3, plot_actions = True)
314
315 # ---- Cell 111 (run 69) ----
316 overall, fnavs, mnavs = ai_trade_performance(env_ti, model_ti4)
317 actions_ti = analyze_actions_taken(env_ti, model_ti4, plot_actions = True)
318
319 # ---- Cell 112 (run 70) ----
320 overall, fnavs, mnavs = ai_trade_performance(env_ti, model_ti5)
321 actions_ti = analyze_actions_taken(env_ti, model_ti5, plot_actions = True)
322
323 # ---- Cell 115 (run 71) ----
324 import yfinance as yf
325 import pandas as pd
326 import pandas_ta as ta
327 import numpy as np
328
329 # download data
330 ticker = yf.Ticker('TM')
331 df_raw = ticker.history(start = '2012-01-01', end = '2017-12-31')
332
333 # prepare the data as within the environment
334 df = df_raw.copy()
335 df = df[['Open', 'High', 'Low', 'Close', 'Volume']]
336 df.loc[:, 'returns'] = np.log(df.Close) - np.log(df.Close.shift(1))
337
338 # this is tomorrow's log-return
339 df.loc[:, 'returns_target'] = df.returns.shift(-1)
340 df.ta.strategy()
341
342 # these variables need to be deleted as they include data from the future
343 # this is called data leakage and creates superior results, however, is not applicable for real-
    life trading
344 df.drop(['DPO_20', 'ISA_9', 'ISB_26', 'ITS_9', 'ICS_26', 'IKS_26'], axis = 1, inplace = True)
345
346 # delete indicators with too many missing values
347 cols_to_drop = df.columns[df.isnull().mean() > 0.05]
348 df.drop(cols_to_drop, axis = 1, inplace = True)
349 # delete rows with missing data
350 df.dropna(inplace = True)
351
352 most_corr = df.corr(method = 'spearman').returns_target.abs().sort_values(ascending = False)
    [1:11].index.to_list()
353 # use this for the use_variables argument when defining an environment
354 most_corr
355
356 # ---- Cell 116 (run 72) ----
357 env_ti_corr = TradingEnvironment(ticker = "TM", start_date = '2012-01-01', end_date = '
    2017-12-31', use_variables = most_corr)
358
359 # ---- Cell 117 (run 73) ----
360 model_ti_corr5 = A2C('MlpPolicy', env_ti_corr, verbose = 0)
361 model_ti_corr5.learn(total_timesteps = 75000)
362

```

```

363 # ---- Cell 118 (run 74) ----
364 model_ti_corr4 = A2C('MlpPolicy', env_ti_corr, verbose = 0)
365 model_ti_corr4.learn(total_timesteps = 75000)
366
367 # ---- Cell 119 (run 75) ----
368 model_ti_corr3 = A2C('MlpPolicy', env_ti_corr, verbose = 0)
369 model_ti_corr3.learn(total_timesteps = 75000)
370
371 # ---- Cell 120 (run 76) ----
372 model_ti_corr2 = A2C('MlpPolicy', env_ti_corr, verbose = 0)
373 model_ti_corr2.learn(total_timesteps = 75000)
374
375 # ---- Cell 121 (run 77) ----
376 model_ti_corr1 = A2C('MlpPolicy', env_ti_corr, verbose = 0)
377 model_ti_corr1.learn(total_timesteps = 75000)
378
379 # ---- Cell 122 (run 78) ----
380 overall, fnavs, mnavs = ai_trade_performance(env_ti_corr, model_ti_corr1)
381 actions_ti_corr = analyze_actions_taken(env_ti_corr, model_ti_corr1, plot_actions = True)
382
383 # ---- Cell 123 (run 79) ----
384 overall, fnavs, mnavs = ai_trade_performance(env_ti_corr, model_ti_corr2)
385 actions_ti_corr = analyze_actions_taken(env_ti_corr, model_ti_corr2, plot_actions = True)
386
387 # ---- Cell 124 (run 80) ----
388 overall, fnavs, mnavs = ai_trade_performance(env_ti_corr, model_ti_corr3)
389 actions_ti_corr = analyze_actions_taken(env_ti_corr, model_ti_corr3, plot_actions = True)
390
391 # ---- Cell 125 (run 81) ----
392 overall, fnavs, mnavs = ai_trade_performance(env_ti_corr, model_ti_corr4)
393 actions_ti_corr = analyze_actions_taken(env_ti_corr, model_ti_corr4, plot_actions = True)
394
395 # ---- Cell 126 (run 82) ----
396 overall, fnavs, mnavs = ai_trade_performance(env_ti_corr, model_ti_corr5)
397 actions_ti_corr = analyze_actions_taken(env_ti_corr, model_ti_corr5, plot_actions = True)
398
399 # ---- Cell 129 (run 83) ----
400 from aif_environment import TradingEnvironment
401 from aif_analysis import ai_trade_performance, analyze_actions_taken
402 from stable_baselines3 import A2C, PPO
403 import numpy as np
404
405 env_train1 = TradingEnvironment(ticker = "TM",
406                               start_date = '2015-01-01',
407                               end_date = '2018-12-31'
408 )
409
410 env_test1 = TradingEnvironment(ticker = "TM",
411                               use_variables = env_train1.data_source.df.columns[1:].to_list(),
412                               scaler = env_train1.data_source.scaler,
413                               start_date = '2012-01-01',
414                               end_date = '2014-12-31'
415 )
416
417 model_train_test1 = A2C('MlpPolicy', env_train1, verbose = 0)
418 model_train_test1.learn(total_timesteps = 75000)
419
420 overall, fnavs, mnavs = ai_trade_performance(env_train1, model_train_test1)
421 overall, fnavs, mnavs = ai_trade_performance(env_test1, model_train_test1)
422
423 actions_training = analyze_actions_taken(env_train1, model_train_test1, plot_actions = True)
424 actions_test = analyze_actions_taken(env_test1, model_train_test1, plot_actions = True)
425

```

```

426 # ---- Cell 130 (run 84) ----
427 env_train2 = TradingEnvironment(ticker = "TM",
428     start_date = '2016-01-01',
429     end_date = '2019-12-31'
430 )
431
432 env_test2 = TradingEnvironment(ticker = "TM",
433     use_variables = env_train2.data_source.df.columns[1:].to_list(),
434     scaler = env_train2.data_source.scaler,
435     start_date = '2013-01-01',
436     end_date = '2015-12-31'
437 )
438
439 model_train_test2 = A2C('MlpPolicy', env_train2, verbose = 0)
440 model_train_test2.learn(total_timesteps = 75000)
441
442 overall, fnavs, mnavs = ai_trade_performance(env_train2, model_train_test2)
443 overall, fnavs, mnavs = ai_trade_performance(env_test2, model_train_test2)
444
445 actions_training = analyze_actions_taken(env_train2, model_train_test2, plot_actions = True)
446 actions_test = analyze_actions_taken(env_test2, model_train_test2, plot_actions = True)
447
448 # ---- Cell 131 (run 85) ----
449 env_train3 = TradingEnvironment(ticker = "TM",
450     start_date = '2014-01-01',
451     end_date = '2017-12-31'
452 )
453
454 env_test3 = TradingEnvironment(ticker = "TM",
455     use_variables = env_train3.data_source.df.columns[1:].to_list(),
456     scaler = env_train3.data_source.scaler,
457     start_date = '2017-01-01',
458     end_date = '2019-12-31'
459 )
460
461 model_train_test3 = A2C('MlpPolicy', env_train3, verbose = 0)
462 model_train_test3.learn(total_timesteps = 75000)
463
464 overall, fnavs, mnavs = ai_trade_performance(env_train3, model_train_test3)
465 overall, fnavs, mnavs = ai_trade_performance(env_test3, model_train_test3)
466
467 actions_training = analyze_actions_taken(env_train3, model_train_test3, plot_actions = True)
468 actions_test = analyze_actions_taken(env_test3, model_train_test3, plot_actions = True)
469
470 # ---- Cell 132 (run 86) ----
471 env_train4 = TradingEnvironment(ticker = "TM",
472     start_date = '2012-01-01',
473     end_date = '2015-12-31'
474 )
475
476 env_test4 = TradingEnvironment(ticker = "TM",
477     use_variables = env_train4.data_source.df.columns[1:].to_list(),
478     scaler = env_train4.data_source.scaler,
479     start_date = '2016-01-01',
480     end_date = '2018-12-31'
481 )
482
483 model_train_test4 = A2C('MlpPolicy', env_train4, verbose = 0)
484 model_train_test4.learn(total_timesteps = 75000)
485
486 overall, fnavs, mnavs = ai_trade_performance(env_train4, model_train_test4)
487 overall, fnavs, mnavs = ai_trade_performance(env_test4, model_train_test4)
488

```

```

489 actions_training = analyze_actions_taken(env_train4, model_train_test4, plot_actions = True)
490 actions_test = analyze_actions_taken(env_test4, model_train_test4, plot_actions = True)
491
492 # ---- Cell 133 (run 87) ----
493 env_train5 = TradingEnvironment(ticker = "TM",
494     start_date = '2016-01-01',
495     end_date = '2019-12-31'
496 )
497
498 env_test5 = TradingEnvironment(ticker = "TM",
499     use_variables = env_train5.data_source.df.columns[1:].to_list(),
500     scaler = env_train5.data_source.scaler,
501     start_date = '2013-01-01',
502     end_date = '2015-12-31'
503 )
504
505 model_train_test5 = A2C('MlpPolicy', env_train5, verbose = 0)
506 model_train_test5.learn(total_timesteps = 75000)
507
508 overall, fnavs, mnavs = ai_trade_performance(env_train5, model_train_test5)
509 overall, fnavs, mnavs = ai_trade_performance(env_test5, model_train_test5)
510
511 actions_training = analyze_actions_taken(env_train5, model_train_test5, plot_actions = True)
512 actions_test = analyze_actions_taken(env_test5, model_train_test5, plot_actions = True)

```

Listing 1: Complete code of the project notebook.